SchoolERP API Reference

Version: 0.1.0
Base URL (local): http://localhost:3002/api/v1
Interactive OpenAPI UI: http://localhost:3002/api/v1/docs

This reference documents the API routes currently implemented in the NestJS API. It is intended for local development and manual testing. The examples use fictional data; replace UUID placeholders with IDs returned by earlier requests.

Authentication and request rules

• Health routes and POST /auth/login and POST /auth/refresh are public. Other routes require a bearer access token unless stated otherwise.
• Log in, copy accessToken, then use Swagger's Authorize button. For an HTTP Bearer scheme, paste the token value; Swagger adds the Bearer prefix.
• JSON request bodies must use double-quoted property names and string values.
• The API validates request bodies. Unknown properties are rejected. Dates below use YYYY-MM-DD.
• Pagination query parameters use page (default 1) and pageSize (default 25, maximum 100).
• :id, :yearId, :classLevelId, :sectionId, and :enrollmentId are UUID path parameters unless the route indicates otherwise.
• Do not share access tokens, refresh tokens, passwords, or real student/guardian data in screenshots or chat.

Recommended manual test order

1. Check health, then log in as the bootstrapped owner.
2. Create an academic year, term, class level, subject, and section.
3. Add the subject to the class curriculum, then to the section.
4. Create teacher accounts and assignments.
5. Create a student and parent account, create/link a guardian, then enroll the student.
6. Try scoped reads with teacher, parent, and student accounts. Use disposable local records for close/end operations.

Create the first owner interactively, if not already done: corepack pnpm --filter @schoolerp/api bootstrap:owner. The command displays a generated temporary password once; the owner must change it at first sign-in. Run it in a private terminal, not a recorded/shared session.

Health

These endpoints are public and have no request body.

Method | Endpoint | Purpose
GET | /health | API status and uptime
GET | /health/db | PostgreSQL connectivity
GET | /health/redis | Redis connectivity

Authentication and user accounts

`POST /auth/login` — public

```
{
  "email": "owner@example.com",
  "password": "Your-Local-Test-Password"
}
```

Returns an access token, refresh token, and user details. Keep both tokens private.

`POST /auth/refresh` — public

```
{
  "refreshToken": "<refreshToken returned by login>"
}
```

`GET /auth/me` — authenticated

No body. Returns the current account, roles, and permissions.

`POST /auth/logout` — authenticated

No body. Revokes the current session. The Authorization: Bearer <accessToken> header is required.

`POST /auth/change-password` — authenticated

```
{
  "currentPassword": "Your-Current-Test-Password",
  "newPassword": "Your-New-Long-Test-Password"
}
```

The new password must contain at least 8 characters. Existing sessions are revoked.

`GET /auth/users` — `user:manage`

No body. Optional query: ?page=1&pageSize=25.

`POST /auth/users` — `user:manage`

```
{
  "email": "teacher@example.com",
  "displayName": "Test Teacher",
  "role": "TEACHER"
}
```

Valid roles: OWNER, PRINCIPAL, ADMIN, ADMISSIONS_OFFICER, ACCOUNTANT, TEACHER, PARENT, STUDENT, TRANSPORT_MANAGER. Only an owner can provision privileged owner/admin/principal accounts. The API generates a cryptographically random temporary password and returns it once in the creation response; deliver it securely. The user must change it at first sign-in. New user-chosen passwords must contain at least 8 characters.

`PATCH /auth/users/{id}` — `user:manage`

```
{
  "role": "TEACHER",
  "status": "ACTIVE"
}
```

Both fields are optional. status must be a valid UserStatus value shown by the API's OpenAPI schema.

Academic years

All routes require a bearer token. Reads require academic:read; writes require academic:write.

Method | Endpoint | Body
GET | /academic/years?page=1&pageSize=25 | None
POST | /academic/years | See create example below
PATCH | /academic/years/{id} | Any editable fields from the create example
POST | /academic/years/{id}/activate | None
POST | /academic/years/{id}/close | None

Create year:

```
{
```

```
  "code": "AY-2026",
  "name": "Academic Year 2026-2027",
  "startsOn": "2026-06-01",
  "endsOn": "2027-05-31"
}
```

Only one academic year can be active at a time. Close the current active year before activating another one.

Terms

Requires academic:read for reads and academic:write for writes.

Method | Endpoint | Body
GET | /academic/terms?academicYearId={yearId}&page=1&pageSize=25 | None
POST | /academic/terms | See create example below
PATCH | /academic/terms/{id} | Any editable fields from the create example

Create term:

```
{
  "academicYearId": "<academicYearId>",
  "code": "TERM-1",
  "name": "Term 1",
  "sequence": 1,
  "startsOn": "2026-06-01",
  "endsOn": "2026-10-31"
}
```

Class levels

Requires academic:read for reads and academic:write for writes.

Method | Endpoint | Body
GET | /academic/class-levels?page=1&pageSize=25 | None
POST | /academic/class-levels | See create example below
PATCH | /academic/class-levels/{id} | Any editable fields from the create example

Create class level:

```
{
  "code": "CLASS-5",
  "name": "Class 5",
  "sortOrder": 5,
  "isActive": true
}
```

Sections

Requires academic:read for reads and academic:write for writes.

Method | Endpoint | Body
GET | /academic/sections?academicYearId={yearId}&classLevelId={classLevelId}&page=1&pageSize=25 | None; filters are optional
POST | /academic/sections | See create example below
PATCH | /academic/sections/{id} | Any editable fields from the create example; isActive may also be supplied

Create section:

```
{
  "academicYearId": "<academicYearId>",
  "classLevelId": "<classLevelId>",
```

```
  "code": "A",
  "name": "Class 5 - A",
  "capacity": 40
}
```

Subjects and curriculum

Requires academic:read for reads and academic:write for writes.

Subjects

Method | Endpoint | Body
GET | /academic/subjects?page=1&pageSize=25 | None
POST | /academic/subjects | See create example below
PATCH | /academic/subjects/{id} | Any editable fields from the create example

```
{
  "code": "MATH",
  "name": "Mathematics",
  "isActive": true
}
```

Class curriculum subjects

Method | Endpoint | Body
GET | /academic/years/{yearId}/class-levels/{classLevelId}/subjects | None
POST | /academic/years/{yearId}/class-levels/{classLevelId}/subjects | See example below

```
{
  "subjectId": "<subjectId>",
  "isRequired": true,
  "sequence": 1
}
```

Subjects offered in a section

Method | Endpoint | Body
GET | /academic/sections/{sectionId}/subjects | None
POST | /academic/sections/{sectionId}/subjects | See example below

```
{
  "classSubjectId": "<classSubjectId returned by the curriculum endpoint>"
}
```

Teacher assignments

Requires academic:read for reads and academic:write for writes. The teacher account must be active and have the TEACHER role. Teacher reads are limited to their own current assignments in the active academic year.

Subject teacher assignments

Method | Endpoint | Body
GET | /academic/teacher-assignments?page=1&pageSize=25&teacherUserId={userId}&academicYearId={yearId} | None; filters are optional
POST | /academic/teacher-assignments | See create example below
PATCH | /academic/teacher-assignments/{id}/end | See end example below

Create assignment:

```
{
```

```
  "sectionSubjectId": "<sectionSubjectId>",
  "teacherUserId": "<teacherUserId>",
  "startsOn": "2026-06-01",
  "endsOn": null
}
```

End assignment:

```
{
  "endsOn": "2026-10-31"
}
```

Class teacher assignments

Method | Endpoint | Body
GET | /academic/class-teacher-assignments?page=1&pageSize=25&teacherUserId={userId}&academicYearId={yearId} | None; filters are optional
POST | /academic/class-teacher-assignments | See create example below
PATCH | /academic/class-teacher-assignments/{id}/end | Same end example above

Create class teacher assignment:

```
{
  "sectionId": "<sectionId>",
  "teacherUserId": "<teacherUserId>",
  "startsOn": "2026-06-01",
  "endsOn": null,
  "reason": "Assigned for the current academic year"
}
```

Students and guardians

Student routes require a bearer token. Student reads require student:read; student and guardian writes require student:write. Staff can manage student records; teacher, parent, and student reads are relationship-scoped. Guardian contact reads are restricted to authorized staff.

Students

Method | Endpoint | Body
GET | /students?page=1&pageSize=25&search=riya&academicYearId={yearId}&sectionId={sectionId} | None; filters are optional
GET | /students/family-portal-accounts | None; lists active parent accounts available for linking a sibling
GET | /students/{id} | None
GET | /students/{id}/enrollments | None
POST | /students | See create example below
PATCH | /students/{id} | Any of firstName, lastName, dateOfBirth; studentNumber cannot be changed
PATCH | /students/{id}/link-account | See account-link example below
POST | /students/{id}/archive | None; requires no ENROLLED enrollment
POST | /students/{id}/reactivate | None
POST | /students/promotions | See batch promotion example below; owner, principal, or admin role required

Create student:

```
{
  "studentNumber": "STU-2026-001",
  "firstName": "Riya",
  "lastName": "Sharma",
  "dateOfBirth": "2016-08-14"
}
```

Create a student and a parent/guardian family login in one transaction. The login has the PARENT role and opens the Parent dashboard for the linked child. Use this same family account for siblings by choosing LINK_EXISTING and the existing parent userId.

```
{
  "studentNumber": "STU-2026-002",
  "firstName": "Aarav",
  "lastName": "Sharma",
  "dateOfBirth": "2020-08-14",
  "familyPortal": {
    "action": "CREATE",
    "guardianFirstName": "Amit",
    "guardianLastName": "Sharma",
    "relationship": "Father",
    "email": "amit.sharma@example.com",
    "phone": "+91 9876543210"
  }
}
```

Link another child to an existing family login:

```
{
  "studentNumber": "STU-2026-003",
  "firstName": "Anaya",
  "lastName": "Sharma",
  "familyPortal": {
    "action": "LINK_EXISTING",
    "relationship": "Mother",
    "userId": "<existing parent account userId>"
  }
}
```

Student creation, guardian linking, and new parent account creation commit or roll back together. The API generates a cryptographically random temporary password for a new family account and returns it once in the creation response. Deliver it securely; the user must change it at first sign-in. Linking an existing family account does not generate or reveal a password.

Archive a student without deleting their enrollment history:

```
POST /students/{id}/archive
```

An active ENROLLED enrollment must be closed before archiving. Reactivate an archived record with POST /students/{id}/reactivate.

Promote students from a closed source year. Every source enrollment must be COMPLETED; select one active target-year section per student. The batch is limited to 100 students. Each student is processed transactionally, failures are returned per student, and resubmitting a completed row returns ALREADY_PROMOTED without making a duplicate enrollment.

```
{
  "sourceAcademicYearId": "<closed-source-year-uuid>",
  "targetAcademicYearId": "<later-open-year-uuid>",
  "enrollmentDate": "2027-04-01",
  "students": [
    {
      "studentId": "<student-uuid>",
      "sectionId": "<active-target-section-uuid>"
    }
  ]
}
```

Update student:

```
{
  "lastName": "Verma",
  "dateOfBirth": "2016-08-14"
}
```

Link student login account:

```
{
  "userId": "<active userId with the STUDENT role>"
}
```

Guardians

Method | Endpoint | Body
GET | /students/{studentId}/guardians | None
POST | /students/{studentId}/guardians | See create-and-link example below
PATCH | /students/{studentId}/guardians/{linkId} | Relationship and/or primary-contact status
POST | /students/{studentId}/guardians/{linkId}/end | None; retains relationship history and revokes this relationship's parent access
PATCH | /guardians/{guardianId}/link-account | See account-link example below

Update an existing guardian relationship:

```
{
  "relationship": "Mother",
  "isPrimaryContact": true
}
```

Ending a relationship records its end time. It does not delete the guardian or the historical link. The parent's student access is evaluated against active guardian relationships.

Create guardian and link to student:

```
{
  "firstName": "Amit",
  "lastName": "Sharma",
  "email": "amit.sharma@example.com",
  "phone": "+91 9876543210",
  "relationship": "Father",
  "isPrimaryContact": true
}
```

Link guardian login account (the user must have the PARENT role):

```
{
  "userId": "<active userId with the PARENT role>"
}
```

Enrollments and section placements

All routes require student:write and a bearer token. Use a valid student, academic year, and section. Enrollment and placement changes are audited; use disposable records for close/transfer testing.

Create enrollment

POST /students/{studentId}/enrollments

```
{
  "academicYearId": "<academicYearId>",
```

```
  "sectionId": "<sectionId>",
  "enrollmentDate": "2026-06-01"
}
```

Add a placement / section transfer

POST /enrollments/{enrollmentId}/placements

```
{
  "sectionId": "<destinationSectionId>",
  "startsOn": "2026-09-01",
  "reason": "Moved to another section"
}
```

Close enrollment

POST /enrollments/{enrollmentId}/close

```
{
  "status": "COMPLETED",
  "exitDate": "2027-05-31"
}
```

Allowed close statuses: COMPLETED, WITHDRAWN, TRANSFERRED.

Common manual checks

• A protected endpoint without a token should return 401 Unauthorized.
• A valid account without the required permission should return 403 Forbidden.
• Invalid JSON, UUIDs, dates, or extra properties should be rejected by validation.
• Duplicate business codes/numbers and conflicting effective date ranges should be rejected.
• Teacher, parent, and student accounts should only see records allowed by their active assignments or linked relationships.
• There are no physical-delete endpoints in this API slice. Use the documented lifecycle actions.

Route inventory

All paths below are relative to /api/v1. Auth means bearer authentication is required; permission names are listed where applicable.

Method | Route | Access
GET | /health | Public
GET | /health/db | Public
GET | /health/redis | Public
POST | /auth/login | Public
POST | /auth/refresh | Public
GET | /auth/me | Auth
POST | /auth/logout | Auth
POST | /auth/change-password | Auth
GET | /auth/users | user:manage
POST | /auth/users | user:manage
PATCH | /auth/users/{id} | user:manage
GET | /academic/years | academic:read
POST | /academic/years | academic:write
PATCH | /academic/years/{id} | academic:write
POST | /academic/years/{id}/activate | academic:write
POST | /academic/years/{id}/close | academic:write
GET | /academic/terms | academic:read
POST | /academic/terms | academic:write
PATCH | /academic/terms/{id} | academic:write
GET | /academic/class-levels | academic:read

POST | /academic/class-levels | academic:write
PATCH | /academic/class-levels/{id} | academic:write
GET | /academic/sections | academic:read
POST | /academic/sections | academic:write
PATCH | /academic/sections/{id} | academic:write
GET | /academic/subjects | academic:read
POST | /academic/subjects | academic:write
PATCH | /academic/subjects/{id} | academic:write
GET | /academic/years/{yearId}/class-levels/{classLevelId}/subjects | academic:read
POST | /academic/years/{yearId}/class-levels/{classLevelId}/subjects | academic:write
GET | /academic/sections/{sectionId}/subjects | academic:read
POST | /academic/sections/{sectionId}/subjects | academic:write
GET | /academic/teacher-assignments | academic:read
POST | /academic/teacher-assignments | academic:write
PATCH | /academic/teacher-assignments/{id}/end | academic:write
GET | /academic/class-teacher-assignments | academic:read
POST | /academic/class-teacher-assignments | academic:write
PATCH | /academic/class-teacher-assignments/{id}/end | academic:write
GET | /students | student:read
GET | /students/{id} | student:read
GET | /students/{id}/enrollments | student:read
POST | /students | student:write
PATCH | /students/{id} | student:write
PATCH | /students/{id}/link-account | student:write
GET | /students/{id}/guardians | student:write
POST | /students/{id}/guardians | student:write
POST | /students/{id}/enrollments | student:write
PATCH | /guardians/{id}/link-account | student:write
POST | /enrollments/{enrollmentId}/placements | student:write
POST | /enrollments/{enrollmentId}/close | student:write